

Project Report & AI Engineering Blueprint

Team-based social MVP — Next.js 15 (App Router) + Supabase · Senior Full-Stack / AI-Native Tech Lead

Deliverable: #2 README companion + #3 AI Engineering Blueprint **Stack:** Next.js 15 · Supabase (Postgres/Auth/Realtime) · RLS · Vercel-ready

Status: Built & verified (tsc + build + lint green)

Executive summary

A **team-based social media MVP** where the tenant is a **team, not a person**: one user belongs to exactly one team, there are no individual profiles, and every action — posting, following, messaging — happens under the team identity. **All authorization is enforced in the database via Row-Level Security (RLS)**; the app layer is never the security boundary.

9 / 9

FUNCTIONAL AREAS MET

85

FILES COMMITTED

7 + 3

SQL MIGRATIONS + PGTAP TESTS

3

INDEPENDENT AI TOOLS

The build followed a **plan-first, research-grounded, multi-agent** workflow, and every AI artifact was treated as an untrusted draft. **Three review passes** (one on the AI's plan, one a 3-critic adversarial pass, one over the built code) caught **real shipping-blockers** — catalogued in §6. The result is green on `tsc` + `next build` (13 routes) + `next lint`.

Honest status. Required functional scope, the README, and the AI Engineering Blueprint are **complete and verified**. Outstanding: (1) push to a private GitHub repo (needs the candidate's account + inviting `mehmetduvarci2011@gmail.com`); (2) the *optional* Vercel deploy and video. Verification was done at the type-check / build / lint / pgTAP-authorized / adversarial-review level — a live end-to-end run needs a provisioned Supabase project (env + Auth Hook), documented in the README.

1. Scope coverage — against the case brief

Each requirement area mapped to where it lives in the code. "Met" = implemented, wired end-to-end, and compiles in the green build.

AREA	REQUIREMENT (FROM THE BRIEF)	STATUS	WHERE
1. Auth & session	Email+password, Google OAuth (PKCE), persistent session, clear logged-in/out, user→team association after login	Met	<code>app/(auth)</code> , <code>auth/callback</code> , <code>middleware.ts</code> , <code>site-header</code>
2. Team model	One user → one team, shared permissions, public/private, roles out of scope, team seeded at signup, content team-scoped	Met	<code>handle_new_user</code> trigger, <code>profiles.id</code> PK=FK, RLS
3. Posting	Any member posts as the team; posts team-owned; text + created timestamp; async/error handling	Met	<code>actions/posts.ts</code> , <code>composer</code> , <code>post-card</code>
4. Follow	No self-follow; one-directional; public=immediate; private=request→approve/reject; follow/unfollow; requests stored	Met	<code>actions/follows.ts</code> , <code>app/requests</code> , <code>follows</code> table
5. Messaging	Team→team, any member acts for team, no self-message, start/send/receive, list ordered by newest message, persisted	Met	<code>actions/messages.ts</code> , <code>get_inbox</code> , Realtime <code>message-thread</code>
6. Home feed	Public posts to all incl. logged-out; private only to approved followers; newest-first; clean querying	Met	<code>app/page.tsx</code> , <code>get_feed</code> RPC, anon RLS
7. Data model & security	Act only for own team; all entities scoped; private visibility enforced; schema+RLS explained; no over-abstraction	Met	<code>migrations/0005_rls.sql</code> , <code>docs/plan/02</code> , pgTAP
8. Architecture (README)	High-level architecture, ERD, key trade-offs/alternatives, risks/mitigations	Met	<code>README.md</code> §8/§6.1/§14/§15 (Mermaid)
9. Structure & quality	Next.js App Router, Supabase, clean async/error, reasonable folders, readable code, modern coherent UI	Met	whole repo; <code>ActionState</code> convention; dark UI kit

Deliverables

DELIVERABLE	STATUS	NOTES
1. GitHub repository	Pending push	Committed locally (initial commit, 85 files). Push to a private repo + invite <code>mehmetduvarci2011@gmail.com</code> ; keep private ≥30 days.
2. README.md	Met	Setup, schema, RLS summary, architecture, ERD, assumptions/trade-offs, known limitations, what-to-improve.
3. AI Engineering Blueprint	Met	This document + <code>docs/AI_ENGINEERING_BLUEPRINT.md</code> + <code>CLAUDE.md</code> + <code>docs/plan/</code> + memory files.
4. Live deployment	Optional	Not deployed. Vercel runbook in the README; the app is Vercel-ready.
5. Video walkthrough	Optional	Not recorded.

2. AI tools used — who does what, and why

A deliberate multi-model split: each model is used where its failure mode is cheapest.

TOOL	ROLE	WHY THIS TOOL
Claude Code (Opus 4.8)	Primary agentic driver. Wrote migrations, RLS, Server Actions, components; ran <code>tsc / build / lint</code> ; orchestrated the multi-agent fan-out and the adversarial review.	Terminal-native agent that edits files <i>and</i> runs the toolchain in one loop, so generated SQL/TS is compiled and reviewed — not pasted blind. Re-reads a checked-in ruleset every turn.
Gemini (3.5 Flash)	First-pass architecture sketch: ERD, DDL draft, RLS sketch, middleware, action skeletons.	Fast, long-context brainstorming for the <i>shape</i> of the system — cheap to iterate before committing engineering time. Treated as a proposal to be audited.
NotebookLM	Grounded field research: case PDF + curated Supabase/Next.js docs as sources, answering targeted questions on the decisions an LLM gets subtly wrong.	Answers are citation-bound to uploaded sources , sharply lowering hallucination risk on the risky decisions (RLS recursion, JWT freshness, anon NULL-comparison, keyset vs OFFSET).

Why multi-model, not one model end-to-end: Gemini diverges fast (great breadth, weak correctness) → the throwaway first draft. NotebookLM is conservative and cited (great correctness, narrow) → grades the risky decisions. Claude Code closes the loop by making the code compile and the reviews pass. The handoffs are deliberate.

3. Development workflow

The project ran as a **plan-first, research-grounded, multi-agent fan-out** — not "prompt → paste → hope."



Why plan-then-implement
A 3-day budget rewards not redoing work. A pinned plan + a shared naming/interface contract means independently-generated sections converge instead of drifting — the 5 parallel feature agents wrote **disjoint files that compiled together on the first integration pass**, with zero interface renames.

Where the value is
Two stages: **stages 4 & 6** (the plan is never trusted — it's diffed against grounded research and independent critics) and **stage 8** (the *code* gets the same untrusted-draft treatment). Most defects were caught by the toolchain and the critics, not the author.

4. Agentic rulesets & instruction files

Real, checked-in artifacts (the brief's "examples of useful files") — not prose.

ARTIFACT	PURPOSE
CLAUDE.md	The operating ruleset re-read every turn: product invariants, locked decisions, hard coding rules (<code>await createClient()</code> , upsert-not-insert for conversations, column-only follow updates, the <code>ActionState</code> shape, <code>revalidateTag</code>), authoritative shared naming, and a "definition of done" + adversarial self-review checklist. Encodes the prior bugs so the agent can't regress them.
docs/plan/CONTRACTS.md	The integration contract for parallel implementation — exact file ownership, exported signatures, import paths, conventions, RPC names. This is what let 5 agents build in parallel without drift.
docs/plan/00-09	The durable design, one owner per domain; <code>00-overview.md</code> is authoritative and records the canonical resolution of every defect the plan reviews found (with corrected SQL/TS inline).
Project memory files	Persistent cross-session notes (scope, locked decisions, the prior-plan bugs) that survive context resets and keep the agent aligned.

5. Prompting & context strategy

- **Constraint-first prompts.** Every task leads with the invariant ("Tenant = Team, no profiles, RLS is the security boundary") so the model optimizes inside the real box. Generic "build a social feed" prompts were avoided — they produce user-centric schemas that violate the core model.
- **Grounded > generative for risky decisions.** Anything with a correctness cliff was answered by NotebookLM against cited sources first, then handed to the coding agent as a settled decision.
- **Shared naming/interface contract injected into context.** Identical identifiers across plan files + `CONTRACTS.md` → independently-generated sections compile together.
- **Compile-and-review-in-the-loop.** The agent runs `tsc / build / lint` after edits and feeds failures back; then independent critic agents review the result.
- **Adversarial self-review.** A standing instruction forces re-reading the diff as an attacker ("what does `anon` see? can a caller pass another team's id?"), institutionalized as a multi-agent review workflow with a per-finding refute step.

6. How AI output was reviewed & validated — the concrete proof

Three independent review passes, each catching **real** defects. This is the evidence that AI output was *audited*, not trusted.

Pass 1 — review of the Gemini first-pass plan (6 bugs, pre-code)

BUG	WHY IT BREAKS	FIX
<code>onboarded</code> written to <code>teams</code> but middleware read <code>app_metadata</code>	Always <code>undefined</code> → infinite <code>/onboarding</code> redirect	Inject <code>onboarded</code> into the JWT via the Auth Hook; middleware reads the same claim
<code>getOrCreateConversation</code> used a bare <code>.insert()</code>	Throws <code>23505</code> when the conversation already exists	<code>upsert(..., {onConflict, ignoreDuplicates})</code> then select
Claim injection via a trigger writing <code>raw_app_meta_data</code>	Races the token mint; first token can lack <code>team_id</code>	Custom Access Token Auth Hook reads <code>team_id</code> at mint time
Inbox not ordered by latest message	Fails "ordered by newest message timestamp"	<code>LEFT JOIN LATERAL max(created_at)</code>
<code>createClient()</code> not awaited (Next 15)	<code>cookies()</code> is async → auth silently null	<code>await createClient()</code> everywhere
<code>follows</code> UPDATE not column-restricted	An approver could rewrite the FK columns	<code>REVOKE UPDATE; GRANT UPDATE(status)</code>

Pass 2 — 3-critic review of the Claude-authored plan (4 high / 8 med / 9 low)

The same untrusted-draft discipline applied to the comprehensive plan itself. Among the highs:

- A **JWT claim-path mismatch** (readers split between `app_metadata.team_id` and other paths — would break 100% of RLS).
- The **Auth Hook reading 0 rows under RLS** (`supabase_auth_admin` lacked a read policy on `profiles` / `teams` → nobody can log in).
- A **missing `posts.team_id` default** (so `insert({content})` would fail `NOT NULL`).
- A **`get_feed` contract divergence** and **private-team name visibility** gap.

All resolved canonically in `00-overview.md §6` with corrected SQL/TS.

Pass 3 — adversarial review of the built code (4 critic dimensions + per-finding refute step → 5 confirmed, 0 false positives)

SEVERITY	DEFECT	FIX
Security	<code>check_team_follows()</code> (<code>SECURITY DEFINER</code>) kept the default <code>PUBLIC</code> execute grant with no caller guard → anyone, incl. <code>anon</code> , could probe the private follow graph via PostgREST RPC.	Added a caller guard (probing restricted to the caller's own team) + revoked the <code>PUBLIC</code> /anon grant. <i>Notably the verifier's first-suggested fix would have broken the <code>posts</code> RLS policy that calls the function as <code>authenticated</code> ; the implemented fix accounts for that.</i>
Data/UX	After a reject, the surviving <code>rejected</code> row made a re-request collide (<code>23505</code>), swallowed as a false "success" → permanent lockout.	RLS-permitted delete-then-insert of the rejected tombstone (an <code>upsert</code> would hit the absent follower-side UPDATE policy).
Low	Feed slices each capped at <code>PAGE_SIZE</code> then merged → up to 2×, not the global newest N.	Slice merged result to <code>FEED_PAGE_SIZE</code> .
Low	"New posts" pill could show a stale count after a non-click refresh.	Reset the counter when the baseline changes.
Low	<code>completeOnboarding</code> ignored a <code>refreshSession()</code> error → possible bounce back to <code>/onboarding</code> .	Capture the error and surface a retry instead of redirecting on a stale token.

A bug the human caught while transcribing. Hand-writing the migrations surfaced a **dependency-ordering bug** in the plan: `posts.team_id` defaults to `current_user_team_id()` , but the plan created that function two migrations *after* the table — so `CREATE TABLE posts` would fail on a clean `supabase db reset` . Fixed by defining the table-free JWT resolver in `0001` before the tables.

Validation methods, ranked by what actually caught things: (1) grounded-research diff; (2) independent adversarial critics with a refute step (the RPC leak + the re-request lockout); (3) `tsc` / `build` (green before review); (4) requirement traceback. **Lesson encoded into the workflow: AI breadth is excellent; AI correctness must be mechanically verified.**

7. Key decisions — candidate vs. AI

DECISION / ARTIFACT	DRIVER
Core domain model (Tenant=Team, no profiles, public/private, follow-as-request)	Candidate — read from the case; the non-negotiable frame.
"Don't use MakerKit; build minimal"	Candidate — its account/billing/role model fights one-user-one-team + no-over-abstraction.
First-pass ERD / DDL / RLS sketch	AI (Gemini) — speed draft, audited.
Risky-decision research (RLS recursion, JWT claims, anon RLS, caching, inbox sort)	AI (NotebookLM), candidate-framed — candidate wrote the questions.
Catching the bugs (all three passes)	Candidate-led, AI-assisted — the decisive discipline.
Locked corrections (Auth Hook, upsert, await client, column-GRANT, lateral-join inbox, denormalized is_public)	Candidate — final architecture calls.
Implementation (migrations, RLS, actions, components, tests)	AI (Claude Code), candidate-reviewed — under <code>CLAUDE.md</code> constraints, gated by build + adversarial review.

One-line summary: the candidate owns the model, the constraints, and the corrections; AI owns breadth, drafting, and mechanical execution; and every AI artifact is gated by grounded research + a running toolchain + independent adversarial review before it counts.

8. Architecture & data model (summary)

Shape

One Next.js 15 app on Vercel ↔ one Supabase project. The browser never holds privileged credentials: reads are RLS-gated RSC, writes go through Server Actions, and Realtime is the one live edge (messaging). No bespoke API tier.

Tenant resolution

Active `team_id` lives in the JWT `app_metadata`, injected by a **Custom Access Token Auth Hook** — present on the *first* token, so RLS resolves tenant scope with zero DB round-trips and no refresh dance.

Six tables

`teams` (tenant) · `profiles` (auth.users → one team; PK=FK enforces one-user-one-team) · `posts` (team-owned; `is_public` denormalized from the team, trigger-maintained) · `follows` (single table + `status` enum; composite PK; `CHECK(follower≠following)`) · `conversations` (symmetric, `team_a<team_b` + UNIQUE → one row per pair, self-chat impossible) · `messages` (sender is a team).

Security mechanisms that matter

- anon vs authenticated split policies** keep the private predicate out of the anonymous query plan entirely — a logged-out visitor can never be served private content.
- SECURITY DEFINER RPCs** (`get_feed` / `get_inbox` / `get_incoming_follow_requests`) verify `caller == current_user_team_id()`, pin `search_path=''`, and surface private counterpart names safely — while table RLS stays on as defense-in-depth.
- Column-level GRANT UPDATE(status) on follows** makes approve/reject unable to rewrite the FK columns — RLS gates rows, this gates columns.
- pgTAP RLS tests** assert anon-vs-private feed isolation, follow column-lockdown, and conversation isolation.

9. Trade-offs, limitations & what's next

Key trade-offs

- Own minimal setup over MakerKit (its roles/billing fight the model).
- Feed liveness via revalidation + poll pill, not realtime (smaller blast radius).
- Hybrid feed: RLS backstop on `posts` + a fast `get_feed` RPC.
- Lateral-join inbox ordering (zero write-amplification at MVP scale).

Known limitations / next

- Feed liveness is poll-not-push (~15s) for public posts.
- No team switching / roles / read receipts / media (per scope).
- Realtime feed (Broadcast-from-DB), denormalized `last_message_at`, keyset infinite scroll — documented scale paths.
- Broader Playwright E2E wired into CI.

Risks & mitigations (selected)

RISK	MITIGATION
Private posts leak to a shared cache	One dynamic feed route; private slice fetched only after <code>cookies()</code> ; public slice cached under a non-private tag.
<code>get_feed</code> privilege escalation	In-function <code>_viewer_team_id == current_user_team_id()</code> + <code>search_path=''</code> ; RLS backstop on <code>posts</code> .
Anon reaches private data	Dedicated <code>TO anon USING(is_public=true)</code> ; explicit <code>REVOKE</code> on sensitive tables; <code>check_team_follows</code> grant locked down.
Infinite onboarding redirect	<code>onboarded</code> injected into the JWT and read from that same claim; <code>refreshSession()</code> after onboarding.

10. How to run & submission checklist

Run (local)

```
pnpm install → cp .env.example .env.local (fill Supabase keys) → supabase start → supabase db reset (migrations 0001-0007 + seed) → pnpm dev . Then enable the Auth Hook and disable Confirm email (README §4). Gates: pnpm typecheck · pnpm build · pnpm lint · supabase db test .
```

To submit

- Create a **private** GitHub repo; push the existing commit.
- Invite `mehmetduvarci2011@gmail.com` ; keep private ≥30 days.
- (Optional) Vercel deploy → add the live link to the README.
- (Optional) record a 5–10 min walkthrough.